

DOCUMENTATION TECHNIQUE

Cluster Kubernetes K3



K3S

Samet ARI

Introduction

Ce document présente la mise en place complète d'un cluster **Kubernetes K3S** dans le cadre du projet pédagogique de La Plateforme_ Marseille. L'objectif est de couvrir l'ensemble du cycle de vie d'un cluster : installation, déploiement d'applications, haute disponibilité, stockage persistant, gestion de la configuration, sécurité et orchestration avec Helm.

Objectifs du projet

- Comprendre les concepts fondamentaux de Kubernetes
- Installer et configurer un cluster K3S multi-nœuds
- Déployer et gérer des applications conteneurisées
- Mettre en place la haute disponibilité (HA)
- Gérer le stockage persistant avec PersistentVolumes
- Utiliser ConfigMaps, Secrets et RBAC
- Déployer des applications avec Helm

Infrastructure

VM	Rôle	IP Statique	OS
K8S (kubes-01)	Master / Control-plane	192.168.81.21	Debian 12 Bookworm
kubes-01	Worker 1	192.168.81.22	Debian 12 Bookworm
kubes-02	Worker 2	192.168.81.23	Debian 12 Bookworm

Job 01 — Création des VM et installation de K3S

Contexte

Création de 3 machines virtuelles Debian 12 sans interface graphique sur VMware Workstation Pro. Configuration des adresses IP statiques et installation de K3S sur chaque VM en mode standalone (indépendant).

Configuration IP statique (/etc/network/interfaces)

```
auto ens33
iface ens33 inet static
    address 192.168.81.21    # adapter par VM
    netmask 255.255.255.0
    gateway 192.168.81.2
    dns-nameservers 8.8.8.8 1.1.1.1
```

Installation de K3S

```
apt update && apt install -y curl
curl -sfL https://get.k3s.io | sh -
systemctl status k3s
kubectl get nodes
```

Résultat : 3 VM opérationnelles avec K3S installé. Chaque VM dispose de son propre cluster K3S indépendant à ce stade.

Job 02 — Déploiement des applications

Contexte

Déploiement de 3 applications conteneurisées sur chaque VM : **nginx**, **apache (httpd)** et **mariadb**. Exposition via des services NodePort pour accès externe.

Commandes de déploiement

```
kubectl create deployment nginx --image=nginx
kubectl expose deployment nginx --port=80 --type=NodePort

kubectl create deployment apache --image=httpd
kubectl expose deployment apache --port=80 --type=NodePort
```

Déploiement MariaDB (YAML — méthode recommandée)

```
cat <<EOF | kubectl apply -f -
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mariadb
spec:
  selector:
    matchLabels:
      app: mariadb
  template:
    metadata:
      labels:
        app: mariadb
    spec:
      containers:
      - name: mariadb
        image: mariadb
        env:
        - name: MYSQL_ROOT_PASSWORD
          value: 'rootpass'
EOF
```

Problème : Le flag `--env` n'est pas supporté dans cette version de `kubectl`. La définition via YAML est la bonne pratique recommandée.

Résultat

Application	Image	Port	Type de service
nginx	nginx:latest	80	NodePort
apache	httpd:latest	80	NodePort
mariadb	mariadb:latest	3306	NodePort

Job 03 — Création du cluster K3S

Contexte

Transformation des 3 VM indépendantes en un cluster unifié. La **VM K8S** (192.168.81.21) devient le master (control-plane), **kubes-01** et **kubes-02** deviennent des **workers (agents)**.

Récupération du token master

```
# Sur le master (K8S)
cat /var/lib/rancher/k3s/server/node-token
```

Ajout des workers au cluster

```
# Sur kubes-01 et kubes-02 :
systemctl stop k3s
systemctl disable k3s
apt install -y iptables

curl -sfL https://get.k3s.io | \
  K3S_URL=https://192.168.81.21:6443 \
  K3S_TOKEN=<TOKEN> sh -

systemctl status k3s-agent
```

Vérification du cluster

```
kubectl get nodes
```

Nœud	Rôle	Statut	Version
k8s	control-plane	Ready	v1.35.5+k3s1
kubes01	worker	Ready	v1.35.5+k3s1
kubes02	worker	Ready	v1.35.5+k3s1

Point important : Les applications des workers ont disparu lors de la transformation en agents — leurs clusters K3S locaux ont été arrêtés. Les applications du master sont restées intactes. Ce comportement est normal et attendu : les pods appartiennent au cluster, pas à la machine.

Problème : Port 6444 déjà utilisé sur kubes-01 (K3S server toujours actif). Solution : `systemctl stop k3s && systemctl disable k3s` avant d'installer l'agent.

Job 04 — Haute Disponibilité (HA)

Contexte

Suppression de toutes les applications existantes et redéploiement avec des **replicas** pour activer la haute disponibilité. Test de la HA en simulant la panne d'un worker.

Suppression des anciens déploiements

```
kubectl delete deployment nginx apache mariadb
kubectl delete service nginx apache mariadb
```

Redéploiement avec replicas

```
kubectl create deployment nginx --image=nginx --replicas=3
kubectl expose deployment nginx --port=80 --type=NodePort

kubectl create deployment apache --image=httpd --replicas=3
kubectl expose deployment apache --port=80 --type=NodePort
```

Répartition des pods (kubectl get pods -o wide)

Application	Replicas	k8s (master)	kubes01	kubes02
nginx	3	1 pod	1 pod	1 pod
apache	3	1 pod	1 pod	1 pod
mariadb	2	—	1 pod	1 pod

Test HA — Arrêt de kubes-02

```
# Extinction de kubes-02
poweroff # sur kubes-02

# Observation sur le master
kubectl get nodes
kubectl get pods -o wide -w
```

Séquence observée :

- ~40 secondes : kubes-02 passe en statut NotReady
- ~5 minutes : pods en Terminating sur kubes-02
- Nouveaux pods en Pending puis ContainerCreating sur k8s et kubes-01
- Pods Running sur les nœuds restants — aucune intervention manuelle

Résultat : Haute disponibilité validée. Le cluster a automatiquement replanifié les pods après la panne du worker. Le délai de 5 minutes correspond au délai d'éviction par défaut de Kubernetes (node-eviction-timeout).

Job 05 — Stockage Persistant (PV/PVC)

Contexte

Mise en place du stockage persistant pour nginx et MariaDB. Par défaut, les données d'un pod sont perdues à sa suppression. Les **PersistentVolumes (PV)** et **PersistentVolumeClaims (PVC)** permettent de conserver les données indépendamment du cycle de vie des pods.

Création des PVC

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mariadb-pvc
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: local-path
  resources:
    requests:
      storage: 2Gi
EOF
```

StorageClass local-path : K3S intègre nativement local-path — le provisionnement dynamique des volumes est disponible sans configuration supplémentaire. Le PVC reste en Pending jusqu'à ce qu'un pod le monte (WaitForFirstConsumer).

Test de persistance

```
# 1. Écriture des données
kubectl exec -it deployment/mariadb -- mariadb -uroot -pSuperSecret2026 -e \
  "CREATE DATABASE laplateforme; USE laplateforme; \
  CREATE TABLE etudiants (id INT, nom VARCHAR(50)); \
  INSERT INTO etudiants VALUES (1, 'Samet');"

# 2. Suppression du pod
kubectl delete pod -l app=mariadb

# 3. Vérification après recréation automatique
kubectl exec -it deployment/mariadb -- mariadb -uroot -pSuperSecret2026 -e \
  "SELECT * FROM laplateforme.etudiants;"
```

Résultat : Les données (1 | Samet) sont toujours présentes après la suppression et la recréation du pod. La persistance est confirmée.

Job 06 — ConfigMaps

Contexte

Les ConfigMaps permettent de séparer la configuration du code. La modification d'une configuration ne nécessite pas de reconstruire l'image Docker — on met à jour le ConfigMap et on redémarre le pod.

Création du ConfigMap

```
cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: ConfigMap
metadata:
  name: nginx-html
data:
  index.html: |
    <html>
    <body style='background:#1a1a2e;color:#00ff88;text-align:center'>
      <h1>Cluster K3S - La Plateforme</h1>
      <p>Page servie depuis un ConfigMap</p>
    </body>
    </html>
EOF
```

Montage du ConfigMap dans nginx

```
volumes:
- name: nginx-html
  configMap:
    name: nginx-html
volumeMounts:
- name: nginx-html
  mountPath: /usr/share/nginx/html
```

Résultat : La page personnalisée "Cluster K3S - La Plateforme" s'affiche dans le navigateur (<http://192.168.81.21:30493>). La configuration est externalisée — l'image nginx officielle est utilisée sans modification.

Job 07 — Secrets

Contexte

Les Secrets permettent de stocker des données sensibles (mots de passe, tokens, certificats) de façon séparée des manifestes de déploiement. Contrairement aux ConfigMaps, les Secrets sont encodés en base64 et peuvent être chiffrés au repos.

Création du Secret

```
kubectl create secret generic mariadb-secret \
  --from-literal=MYSQL_ROOT_PASSWORD=SuperSecret2026

# Vérification (valeur encodée en base64)
kubectl get secret mariadb-secret -o yaml
```

Utilisation dans le déploiement MariaDB

```
env:
- name: MYSQL_ROOT_PASSWORD
  valueFrom:
    secretKeyRef:
      name: mariadb-secret
      key: MYSQL_ROOT_PASSWORD
```

Critère	ConfigMap	Secret
Usage	Config non-sensible (URLs, fichiers)	Données sensibles (mots de passe, tokens)
Stockage	Texte clair	Base64 (chiffable au repos)
Exemple projet	Page HTML nginx	Mot de passe MariaDB

Résultat : Le mot de passe n'apparaît plus en clair dans le manifeste de déploiement. Connexion testée avec SuperSecret2026 — accès confirmé. Tentative avec l'ancien mot de passe : Access denied (preuve que le Secret est actif).

Job 08 — RBAC (Role-Based Access Control)

Contexte

Le **RBAC (Contrôle d'accès basé sur les rôles)** permet de définir précisément qui peut faire quoi dans le cluster. C'est l'application du principe du moindre privilège, fondamental en cybersécurité.

Composants RBAC

Composant	Ressource créée	Rôle
ServiceAccount	stagiaire	Identité (qui ?)
Role	pod-reader	Permissions (quoi ?)
RoleBinding	stagiaire-pod-reader	Association identité ↔ rôle

Création du Role (lecture seule)

```
cat <<EOF | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: default
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
EOF
```

Test des permissions

```
# Peut lister les pods ? -> yes
kubectl auth can-i list pods --as=system:serviceaccount:default:stagiaire

# Peut supprimer des pods ? -> no
kubectl auth can-i delete pods --as=system:serviceaccount:default:stagiaire

# Tentative de suppression -> Forbidden
kubectl delete pod -l app=nginx --as=system:serviceaccount:default:stagiaire
```

Résultat : Principe du moindre privilège appliqué. Le compte stagiaire peut lister les pods mais reçoit une erreur Forbidden lors d'une tentative de suppression. Le RBAC est opérationnel.

Job 09 — Helm

Contexte

Helm est le gestionnaire de paquets de Kubernetes. Il permet d'installer, personnaliser, mettre à jour et désinstaller des applications complexes via des charts — équivalent de apt pour Debian ou npm pour Node.js.

Installation de Helm

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 | bash
export KUBECONFIG=/etc/rancher/k3s/k3s.yaml
echo 'export KUBECONFIG=/etc/rancher/k3s/k3s.yaml' >> ~/.bashrc
helm version
```

Ajout d'un dépôt et installation

```
helm repo add podinfo https://stefanprodan.github.io/podinfo
helm repo update

# Installation avec personnalisation
helm install mon-app podinfo/podinfo \
  --set ui.message="Cluster K3S - La Plateforme" \
  --set replicaCount=2
```

Cycle de vie complet

```
# Mise à jour (upgrade) -> REVISION 2
helm upgrade mon-app podinfo/podinfo --set replicaCount=2

# Historique
helm history mon-app

# Retour arrière (rollback) -> REVISION 3
helm rollback mon-app 1

# Désinstallation
helm uninstall mon-app
```

Problème rencontré : Le chart Bitnami/apache était inutilisable : images supprimées de Docker Hub en 2025. Diagnostic via kubectl describe pod. Solution : changement de dépôt vers podinfo (Stefan Prodan / CNCF). Leçon : toujours vérifier la pérennité des sources.

Résultat : Cycle complet Helm maîtrisé : install → personnalisation (message visible dans le navigateur) → upgrade (revision 2) → rollback → uninstall. La ligne "Served by" de podinfo démontre le load balancing entre les 2 replicas lors du rafraîchissement de la page.

Pour aller plus loin — K3S vs Docker vs Docker Swarm

Positionnement

Docker est un moteur de conteneurs, pas un orchestrateur. La comparaison pertinente est Docker Swarm vs K3S. Docker est la technologie de base sur laquelle les deux s'appuient.

Critère	Docker	Docker Swarm	K3S (Kubernetes)
Type	Moteur de conteneurs	Orchestrateur léger	Orchestrateur complet
Courbe d'apprentissage	Faible	Moyenne	Élevée
Haute disponibilité	Non	Oui (replicas)	Oui (self-healing avancé)
Stockage persistant	Volumes locaux	Volumes + NFS manuel	PV/PVC dynamique (StorageClass)
Secrets	Limité	Docker secrets	Secrets + RBAC
Contrôle d'accès	Non	Basique	RBAC complet
Gestionnaire de paquets	Non	Non	Helm
Écosystème	Docker Hub	Limité	CNCF — énorme
Standard industrie	Oui	En déclin depuis 2020	Standard actuel

Quand utiliser quoi ?

Scénario	Outil recommandé
Dev local, tests, un seul serveur	Docker seul
Petite équipe, 2-5 serveurs, besoin simple de HA	Docker Swarm
Production moderne, RBAC, écosystème riche	K3S / Kubernetes
IoT, edge computing, ressources limitées	K3S spécifiquement

Retour d'expérience — deux projets comparés

Ayant monté les deux infrastructures (cluster Docker Swarm 4 VM + NFS, et ce cluster K3S 3 nœuds) :

- **Swarm** : installation plus rapide, concepts plus proches de Docker — mais NFS à configurer manuellement, pas de RBAC, pas de Helm
- **K3S** : StorageClass local-path intégrée, RBAC natif, Helm disponible — mais courbe d'apprentissage plus importante (pod, deployment, service, PVC, Secret...)
- K3S est la compétence que le marché demande aujourd'hui